Instrucciones para el alumnado

1. Crea un proyecto con tsc --init y deja activada la opción "strict": true en tsconfig.json.
2. Resuelve cada ejercicio en su propio fichero (ej01.ts, ej02.ts…) y compílalo con tsc (puedes usar --watch).
3. El código debe compilar sin errores y **sin usar any**. Si un dato puede ser «cualquier cosa», piensa qué alternativa segura ofrece TypeScript.
4. Anota tipos en las «fronteras» del código (parámetros, valores de retorno, modelos de datos) y deja que la inferencia haga el resto.
5. Para cada ejercicio, decide tú qué herramienta es la más adecuada: tipos básicos, arrays, tuplas, type, interface, enum, uniones, intersecciones, Record, genéricos, utility types, etc.
6. Comprueba el funcionamiento con console.log y con node sobre el .js generado. También puedes usar el TypeScript Playground.

Enunciados

Ejercicio 1. Notas de un alumno

La secretaría de un instituto necesita una pequeña utilidad para calcular la nota final de un alumno.

1. Guarda en variables el nombre del alumno, su curso, si es repetidor o no, y la lista de sus notas de evaluación.
2. Crea una función calcularMedia que reciba la lista de notas y devuelva la media.
3. Crea una función notaFinal que reciba la lista de notas y, de forma opcional, unos puntos extra por participación (si no se indican, valen 0). Devuelve la media más esos puntos, sin pasar nunca de 10.
4. Crea una función mostrarBoletin que imprima por consola el nombre, el curso y la nota final, y que no devuelva nada.

Ejercicio 2. Carrito de una tienda online

Una tienda online quiere modelar los productos de su catálogo y el carrito de la compra.

1. Cada producto tiene un identificador que **nunca puede cambiar** una vez creado, un nombre, un precio y, solo en algunos casos, un porcentaje de descuento.
2. Una línea del carrito está formada por un producto y la cantidad de unidades.
3. Crea una función precioFinal que devuelva el precio de un producto aplicando su descuento si lo tiene.
4. Crea una función totalCarrito que reciba todas las líneas del carrito y devuelva el importe total.

5. Crea un carrito con al menos tres productos (uno con descuento) y muestra el total. Comprueba que el compilador da error si intentas cambiar el identificador de un producto.

Ejercicio 3. Ruta de un repartidor

Una empresa de mensajería registra la ruta de sus repartidores como una sucesión de puntos GPS. Cada punto está formado **siempre** por dos valores en este orden: latitud y longitud.

1. Guarda la ruta de un repartidor con al menos cuatro puntos.
2. Crea una función distanciaAproximada que reciba dos puntos y devuelva la distancia entre ellos usando la fórmula euclídea $\sqrt{((lat2-lat1)^2 + (lon2-lon1)^2)}$.
3. Crea una función distanciaTotal que reciba la ruta y devuelva la suma de las distancias entre puntos consecutivos.
4. Cada entrega se registra con tres datos fijos y ordenados: el código del paquete, el punto donde se entregó y si el cliente firmó la recepción. Crea dos entregas y muéstralas por consola.
5. Comprueba que el compilador da error si un punto tiene tres coordenadas o si los datos de una entrega están desordenados.

Ejercicio 4. Seguimiento de pedidos

En la web de una tienda, un pedido solo puede estar en uno de estos estados: "pendiente", "pagado", "enviado", "entregado" o "cancelado". Ningún otro texto es válido.

1. Define el estado de un pedido de forma que el compilador rechace cualquier otro valor.
2. Define un pedido con número, nombre del cliente, importe y estado.
3. Crea una función mensajeCliente que, según el estado, devuelva el texto que verá el cliente. Usa un switch y garantiza que, si mañana se añade un estado nuevo y se te olvida tratarlo, **el código deje de compilar**.
4. Crea una función error que reciba un mensaje y lance siempre una excepción. Úsala en una función cancelarPedido que lance el error si el pedido ya está entregado y, si no, cambie su estado a cancelado.

Ejercicio 5. Gestión de incidencias informáticas

El departamento de informática de una empresa registra las incidencias que abren los empleados.

1. La prioridad de una incidencia es uno de estos valores con nombre: Baja, Media, Alta y Crítica, de forma que se puedan comparar numéricamente (Crítica es la mayor).
2. El estado de una incidencia es uno de estos valores con nombre, que se guardan como texto: "ABIERTA", "EN_CURSO", "RESUELTA".
3. Cada incidencia tiene un id, un título, el empleado que la abre, la prioridad, el estado y la fecha de apertura.

4. Crea las funciones:
    - filtrarPorEstado: devuelve las incidencias que están en un estado concreto.
    - urgentes: devuelve las incidencias con prioridad Alta o superior que no estén resueltas.
    - cambiarEstado: cambia el estado de una incidencia, muestra por consola el cambio (estado anterior y nuevo) y devuelve la incidencia actualizada.
5. Prueba las funciones con al menos cuatro incidencias.

Ejercicio 6. Usuarios de una biblioteca

La aplicación de la biblioteca municipal tiene dos tipos de usuarios con muchos datos en común.

1. Todo usuario tiene: número de carné, nombre, correo electrónico (opcional) y fecha de alta, que **no puede modificarse** después de crearse.
2. Un socio es un usuario que además tiene la lista de títulos que tiene prestados y un método saludar que imprime un saludo con su nombre.
3. Un bibliotecario es un usuario que además tiene su turno (mañana o tarde, y nada más) y la lista de permisos que tiene.
4. Reutiliza la definición de usuario: no repitas sus propiedades en socio ni en bibliotecario.
5. Crea una función puedePrestar que reciba un socio y devuelva si puede llevarse otro libro (el máximo son 3 préstamos).
6. Crea una función contacto que reciba cualquier usuario y devuelva su correo o, si no lo tiene, el texto "Sin correo".

Ejercicio 7. Stock de un almacén

El almacén de una ferretería guarda las existencias en un objeto cuyas claves son los códigos de producto ("TORN-01", "MART-02"…) y cuyos valores son las unidades disponibles. Los códigos no se conocen de antemano: se van añadiendo.

1. Crea el objeto de stock vacío y añade varios productos.
2. Crea una función reponer que sume unidades a un código (si el código no existe, lo crea).
3. Crea una función vender que reste unidades. Si no hay suficientes, debe devolver false y no tocar el stock; si hay, devuelve true. Es decir, la función devuelve un valor lógico.
4. Crea una función informe que reciba el stock y un mínimo opcional (por defecto 5) y devuelva los códigos de los productos que están por debajo de ese mínimo.
5. Crea una función buscarUnidades que reciba un código y devuelva el número de unidades si existe o el texto "No existe" si no existe.

Ejercicio 8. Datos que llegan de un formulario

Una aplicación recibe los datos de un formulario de registro desde una librería externa. Esa librería no nos garantiza nada: el valor de cada campo puede ser un texto, un número, un booleano, null…

1. Guarda en variables varios valores «de origen desconocido» **sin perder la seguridad de tipos** (no se permite usar any).
2. Crea una función normalizarNombre que reciba un valor desconocido y, solo si es un texto, lo devuelva sin espacios al principio y al final y en mayúsculas. En otro caso devuelve "NOMBRE INVÁLIDO".
3. Crea una función leerEdad que reciba un valor desconocido y devuelva la edad como número si es un número, o si es un texto que representa un número (por ejemplo "25"). En otro caso devuelve null.
4. La librería también devuelve el objeto completo del formulario. Tú sabes que siempre trae usuario y email. Define esa forma y convierte el objeto recibido a ella para poder acceder a sus propiedades con autocompletado.
5. Explica en un comentario por qué la conversión del punto 4 es más arriesgada que las comprobaciones de los puntos 2 y 3.